

Práctica 06.

Bloque Administración Linux.

Gestionando usuarios



Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera

Copyright:



Cubre el objetivo del Linux Professional Institute (LPI) 1.107.1 Qué es una cuenta de usuario y cómo son los nombres de usuario. La cuenta root. Qué son los grupos en linux y para qué se utilizan. Grupo primario y grupos secundarios. Identificadores de Usuario UIDs y de grupo GIDs. Los ficheros `/etc/passwd` `/etc/shadow` `/etc/login.defs` `/etc/group`. Los ficheros de configuración `.profile`, `.bash_profile` y `.bashrc`. Configurar cuentas: crear, modificar y eliminar cuentas. Configuración de contraseñas, la orden `chage`. Diferencia entre las órdenes `su` y `sudo`. Configurar grupos: crear, modificar y eliminar grupos

Contenidos

Contenidos.....	1
1. Introducción.....	2
2. Usuarios y grupos.....	2
3. Conceptos Importantes.....	3
3.1. Nombres de Usuario.....	3
3.2. Grupos.....	4
3.3. UIDs y GIDs.....	5
3.4. El archivo <code>/etc/passwd</code> y archivos con información de usuarios y grupos.....	7
4. Configurando cuentas de usuario.....	11
4.1. Añadir usuarios.....	11
4.2. Modificar cuentas de usuarios.....	14
4.3. Eliminar usuarios.....	19
5. La órdenes <code>su</code> y <code>sudo</code>.....	20
6. Configurando grupos.....	23
6.1. Añadir grupos.....	23
6.2. Modificar grupos.....	24
6.3. Eliminar grupos.....	26
Ejemplos Gestión de Cuentas.....	27
Ejercicios.....	29
Preguntas frecuentes.....	30
Usuario root ¿Cómo pasar a ser usuario root?.....	30

Cuenta root ¿Es necesario activar la cuenta root en Ubuntu?.....	30
Cuenta en Ubuntu: Activar la cuenta root en Ubuntu.....	31
Usuario root ¿Cómo deshabilitar el usuario root?.....	32

1. Introducción

Linux es un **sistema multiusuario**. Una de las primeras tareas de **administración** debe ser la de gestionar los **usuarios del sistema**. En Linux existen dos tipos o perfiles de usuarios:

- el **administrador**, o **'root'**, y
- los **usuarios finales**.

En todo sistema Unix existe un **usuario administrador del sistema (root)**, que controla el funcionamiento completo del sistema. Los cambios o modificaciones realizados por el administrador pueden causar inconvenientes en la seguridad del sistema completo, se debe utilizar `root` solo cuando sea estrictamente necesario llevar a cabo una tarea de administración, y cuando terminemos, debemos volver a trabajar con nuestro usuario normal.

La **administración de usuarios** en Linux es la referida a la **creación y configuración de las cuentas de usuarios, grupos de usuarios**.

2. Usuarios y grupos

Linux es un sistema multiusuario que usa **cuentas de usuario** para identificar a los distintos usuarios del ordenador.

Una **cuenta de usuario** es un conjunto de estructuras de datos y procedimientos que contienen y gestionan toda la información necesaria para distinguir a dos usuarios distintos dentro del sistema.

Una **cuenta de usuario** es una **identidad creada para una persona en una computadora o sistema informático**. También se pueden crear cuentas de usuario para las entidades de la máquina, como **cuentas de servicio** para ejecutar programas, **cuentas del sistema** para almacenar archivos y procesos del sistema y cuentas de administrador y raíz para la administración del sistema.

3. Conceptos Importantes

A continuación se explican algunos conceptos importantes sobre:

- los **Nombres de Usuario**,

- los **Grupos**,
- los **UIDs y GIDs** y
- el archivo `/etc/passwd` que guarda las cuentas de usuario.

3.1. Nombres de Usuario

Cada usuario de un sistema Linux tiene un **nombre de usuario único**. Es el que se usa para acceder al sistema. Es un **nombre simbólico**, con sentido para el administrador del sistema, que le facilita el trabajo, aunque **internamente**, un usuario se representa mediante un número, su **UID (*User Identifier*)**, el **número de identificación de usuario**.

Linux es muy **flexible sobre los nombres de usuario**. La mayoría de las versiones de Linux dan soporte para nombres de usuario que consisten en cualquier combinación de **letras** (mayúsculas y minúsculas), **números** y muchos de los **signos de puntuación** (incluyendo puntos (.), guion bajo (_) y espacio ()), son por tanto, **nombres válidos**. Sin embargo, **aunque sean válidos, se recomienda que como nombre de usuario se usen sólo letras y números** (y como separador el guion bajo o el punto) para evitar problemas con algunas utilidades, se deberían seguir estos consejos:

- **Utilizar caracteres permitidos:** Puedes usar letras (a-z), números (0-9), guion bajo (_) y punto (.).
- **Todos los nombres de usuario deben comenzar con una letra.** Así el nombre de usuario `45u` no está permitido, pero el nombre `u45` sí.
- **Un nombre de usuario puede contener hasta 32 caracteres.** Sin embargo, dado que algunas utilidades truncan los nombres de usuario mayores de 8 caracteres, se recomienda usar nombres de usuario que como máximo tengan 8 caracteres. Este **consejo debido a la cantidad de usuarios es imposible de contemplar, ahora todas las instituciones tienen nombres más largos**.
- **Linux distingue mayúsculas de minúsculas**, por lo tanto, los nombres de usuario `juan` y `Juan` son distintos. Mezclar nombres en mayúsculas y minúsculas puede dar lugar a confusión. Por tanto, **se recomienda usar sólo nombres con letras en minúscula**. Nombres típicos en un sistema Linux son: `facr`, `fconde`, `fconde01`, `f_conde` o `f.conde` para mostrar sólo algunas variaciones posibles.

La mayoría de los sitios desarrollan un método estándar para crear nombres de usuario, como utilizar la primera inicial o iniciales del nombre de pila y el apellido. Crear y seguir un sistema de este tipo puede ayudar a localizar una cuenta que pertenezca a un individuo en concreto.

Los usuarios actualmente definidos en el sistema se gestionan mediante un archivo denominado `/etc/passwd`, y en la mayoría de sistemas Linux actuales, sus contraseñas, mediante el archivo `/etc/shadow`.

3.2. Grupos

Linux usa los **grupos** como un medio para organizar a los usuarios. Hay que tener muy presente que **los grupos NO son cuentas**, sino un medio de **organizar conjuntos de cuentas**.

La principal finalidad de los grupos es aumentar la seguridad del sistema. **Cada fichero en un sistema Linux está asociado con un grupo específico** y se **pueden asociar permisos al grupo**, de forma que todos los usuarios de ese grupo tienen esos permisos. Linux proporciona acceso a la mayoría de los dispositivos hardware a través de ficheros, así que se puede controlar el acceso de los usuarios al hardware, a través de los permisos que se asocian a esos ficheros.

Un grupo puede contener a todos los usuarios del sistema, a ninguno, o a cualquier cantidad intermedia de usuarios. La pertenencia de los usuarios a los grupos se controla mediante el archivo `/etc/group`. Este archivo contiene una lista de todos los grupos definidos en el sistema y los usuarios que pertenecen a cada grupo. **Un usuario puede** (de hecho lo hace) **pertenecer a más de un grupo**.

Cada usuario tiene un grupo por defecto o grupo primario. Cuando un usuario accede al sistema, se le asocia ese grupo primario, de forma que cuando crea un archivo o ejecuta un programa, ese archivo y ese programa están asociados a un único grupo, el grupo primario del usuario.

Esto no significa que el usuario no pueda pertenecer también a otros grupos. Así, por ejemplo, si otro archivo está asociado a otro grupo, al que también pertenece el usuario, éste puede acceder a dicho archivo, siempre y cuando el archivo tenga permisos que permiten el acceso para los miembros de su grupo.

Un usuario **puede crear archivos o ejecutar programas que tengan asociado otro grupo** distinto de su grupo primario. Para ello, primero debe elegir otro grupo **de aquellos a los que pertenece** con la orden `newgrp`.

Suponiendo que existe un usuario llamado `fconde`, cuyo grupo primario es `fconde` (es lo habitual en Ubuntu, se nombra el grupo primario con el mismo nombre de usuario por defecto), pero que también es administrador del sistema, y por tanto, también pertenece al grupo `adm`, la siguiente orden es correcta:

```
fconde@iMac21:~$ newgrp adm
```

A partir de ese momento, cada archivo que cree el usuario `fconde` ya no tendrá como grupo el grupo `fconde`, sino el grupo `adm`.

Si el usuario especifica en la orden `newgrp`, un grupo al cual no pertenece, se le pedirá una contraseña, la contraseña del grupo (más adelante se verá cómo establecer estas contraseñas).

Como hemos visto, **los grupos son un mecanismo muy eficaz para permitir que un conjunto de usuarios trabajen sobre un conjunto de archivos, sin que el resto de usuarios de la máquina pueda acceder a ellos**. Así por ejemplo, se puede establecer un grupo por cada clase, o por cada proyecto, o por cada departamento de una empresa.

Un **usuario puede pertenecer a dos o más grupos**, por ejemplo, un estudiante que pertenezca a varias clases o un empleado que trabaje en dos proyectos. De esta forma, tendrá acceso a los archivos de esos grupos según los permisos que se establezcan para ese grupo.

Un usuario **debe** pertenecer al menos a un grupo, su **grupo primario**, tan es así que **si un usuario no pertenece a ningún grupo, no puede hacer login en el sistema**.

3.3. UIDs y GIDs

Como se ha mencionado anteriormente, los nombres de usuario y de grupo, son nombres simbólicos que ayudan en su tarea al administrador del sistema. Sin embargo, internamente, Linux define usuarios y grupos mediante números llamados **user ID (UID)** y **group ID (GID)** respectivamente.

Cuando un administrador usa un nombre de usuario o de grupo, el sistema Linux usa los archivos `/etc/passwd` y `/etc/group`, para localizar dicho nombre y averiguar cuál es el UID o el GID asociado, y luego usa esos números al ejecutar órdenes.

- `/etc` Contiene archivos de configuración de todo el sistema y bases de datos del sistema; el nombre representa etcétera. Originalmente también contenía "utilidades de mantenimiento peligrosas" como `init`, pero estas se han movido típicamente a `/sbin` o en otro lugar.

En este apartado, vamos a estudiar cómo Linux asigna números a usuarios y grupos, lo que será muy útil si hay problemas y sobre todo si se deben editar los archivos `/etc/passwd` y `/etc/group` manualmente.

Las distribuciones de Linux **reservan los primeros 100 identificadores de usuario y grupo (0-99) para uso del sistema**. El más importante de todos es el **ID 0** que se asigna al `root` (tanto el usuario como el grupo). El resto de IDs del sistema no están completamente estandarizados en todas las distribuciones de Linux.

Así por ejemplo, el `UID 2` y `GID 2` que corresponden con el usuario y grupo `daemon` en Red Hat y SUSE, en Debian y Ubuntu corresponden con el usuario y grupo

Por eso es preferible **referirse a usuarios y grupos mediante su nombre simbólico** (que sí es igual en las distintas distribuciones Linux) que mediante su ID (que puede variar).

Estos IDs corresponden a los que se denomina en linux **usuarios especiales**.

Ejemplos: `bin`, `daemon`, `adm`, `lp`, `sync`, `shutdown`, `mail`, `operator`, `squid`, `apache`, etc.

No tiene todos los privilegios del usuario `root`, pero dependiendo de la cuenta asumen distintos privilegios de `root`. **No tienen contraseñas** pues son cuentas que **no están diseñadas para iniciar sesiones** con ellas. También se les conoce como cuentas de "no inicio de sesión" (`nologin`). Se crean (generalmente) automáticamente al momento de la instalación de Linux o de la aplicación.

A partir del identificador 100, esos IDs están disponibles para usuarios y grupos ordinarios, aunque la mayor parte de distribuciones **reservan hasta el ID 500 o incluso hasta el 1000 (como es el caso de Ubuntu) para propósitos especiales**. Por lo tanto, al primer usuario "normal" del sistema se le asigna el ID 500 o 1000 (**1000 en el caso de Ubuntu**). Estos límites se definen en el archivo `/etc/login.defs`, en concreto en las constantes `UID_MIN`, `UID_MAX` y `GID_MIN`, `GID_MAX`.

Para mostrar estos valores puedes ejecutar la siguiente orden:

```
$ grep -E '^(UID|GID)_(MIN|MAX)' /etc/login.defs | grep -v '#'
```

```
UID_MIN          1000
UID_MAX          60000
GID_MIN          1000
GID_MAX          60000
```

Cuando se crean nuevos usuarios, por lo general, el sistema localiza el siguiente ID más alto no utilizado, así por ejemplo, la segunda cuenta de usuario que se cree tendrá UID y GID 1001, la siguiente 1002, etc. Cuando se elimina una cuenta, el ID asociado podría reutilizarse, sin embargo, el sistema no suele hacerlo si todavía existen cuentas con IDs mayores que el eliminado, quedando un hueco en la secuencia de IDs usados.

Por ejemplo, si se crean las cuentas con IDs 1000, 1001 y 1002; a continuación se elimina la cuenta con ID 1001; y por último se crea una nueva cuenta de usuario, el sistema le asignará ID 1003 y no el 1001 aun a pesar de que está libre.

Se desperdician IDs, pero no suele ser un problema, ya que el número máximo de IDs que pueden asignarse en sistemas Linux con kernel versión 2.2.x es de 65536, y en kernel versión 2.4.x de 4.2 miles de millones. Lo que suele ser más que suficiente para el número habitual de usuarios en un sistema Linux.

Borrar una cuenta de usuario no siempre lleva asociado borrar todos los archivos asociados a esa cuenta. Así si se reutiliza el ID del usuario al que pertenecía la cuenta borrada, automáticamente todos sus archivos no borrados pasan a ser propiedad del nuevo usuario, lo que puede tener consecuencias negativas.

La orden id

La orden `id` muestra el UID y el GID del usuario especificado, además de sus grupos secundarios. Si no se especifica ningún usuario se refiere al usuario que ejecuta la orden. Las opciones se utilizan para mostrar una parte de esta información.

```
id [OPCIONES] [NOMBRE_DE_USUARIO]
```

```
$ id
```

```
uid=1000(lina) gid=1000(lina)
grupos=1000(lina),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),12
2(lpadmin),135(lxd),136(sambashare)
```

```
$ id root
```

```
uid=0(root) gid=0(root) grupos=0(root)
```

3.4. El archivo `/etc/passwd` y archivos con información de usuarios y grupos

El archivo `/etc/passwd`

Toda la información relativa a los usuarios del sistema (a excepción de su contraseña) se encuentra en el archivo `/etc/passwd`.

En ese archivo hay una entrada por cada usuario registrado, cada una con 7 **campos separados** por el símbolo "**dos puntos**" :

```
fconde:x:1000:1000:Francisco de Asís Conde Rodríguez,,,:/home/fconde:/bin/bash
```

1. El primer campo es el **nombre de usuario**. En este ejemplo `fconde`. Es el nombre simbólico que le asignó el administrador al crear su cuenta.
2. El segundo campo (`x`) indica que la contraseña se encuentra cifrada en el archivo `/etc/shadow`.
3. El tercer campo es el **UID del usuario**. En este caso `1000`.
4. El cuarto campo es el **GID del usuario**. En este ejemplo `1000`. Es el identificador del grupo primario o grupo por defecto del usuario.
5. El quinto campo es lo que se conoce como **campo Gecos**. Este nombre proviene de **GCOS** (**Sistema Operativo Comprensivo General**) es una familia de sistemas operativos orientados hacia computadoras centrales (mainframes). Su primera versión fue desarrollada por General Electric en 1962; originalmente designado **GECOS** (*the General Electric Comprehensive Operating Supervisor*). Este campo incluye información sobre el usuario que ayuda a identificarlo. Normalmente es el nombre completo del usuario (Francisco de Asís Conde Rodríguez). Parte de esta información se muestra cuando se pide información sobre un usuario con la orden `finger`. (Vease el apartado añadir usuarios).

6. El sexto campo es el directorio raíz del usuario. Su directorio de trabajo por defecto (/home/fconde).
7. El séptimo campo es la shell que este usuario prefiere tener por defecto. En el ejemplo la shell bash, (/bin/bash). En el archivo /etc/shells se indican los shells permitidos, es decir, los que están instalados en el sistema

El archivo /etc/group

/etc/group es un archivo legible para todos (world-readable) que contiene una lista de grupos, cada uno en una línea separada:

```
developer:x:1002:  
adm:x:4:syslog,procopia
```

Cada línea constante de **4 campos** delimitados por "dos puntos":

1. **Group Name** El nombre del grupo.
2. **Group Password** La contraseña cifrada del grupo (o una x si se usan contraseñas ocultas).
3. **Group ID (GID)** El número de identificación asignado al grupo en el sistema.
4. **Member list** Una lista delimitada por comas de los usuarios que pertenecen al grupo, excepto aquellos para quienes este es el grupo primario. Por ejemplo, el grupo adm, tiene como miembros a syslog y procopia.

El archivo /etc/shadow

/etc/shadow es un archivo legible solo por usuarios root y usuarios con privilegios de root. Además contiene las contraseñas cifradas de los usuarios separadas por una línea.

Esta información aparecería en una sola línea separado cada campo por : pero se muestra aquí en varias líneas:

```
frank:$6$i9gjM4Md4MuelZCd$7jJa8Cd2bbADFH4dwtfvTvJLOYCCCBf/.jYbK1IM  
Yx7Wh4fErXcc2xQVU2N1gb97yIYaiqH.jjJammzof2Jfr/:18029:0:99999:7:::
```

Cada línea consta de **9 campos** delimitados por "dos puntos":

1. **Username:** El nombre utilizado cuando el usuario inicia sesión en el sistema (frank).
2. **Encrypted password:** La contraseña cifrada del usuario (si el valor es un símbolo de exclamación !, La cuenta está bloqueada).
3. **Date of last password change:** La fecha del último cambio de contraseña, como número de días desde 01/01/1970, en el ejemplo (18029). Un valor de 0 significa que el usuario debe cambiar la contraseña en el siguiente acceso.
4. **Minimum password age:** El número mínimo de días después de un cambio de contraseña que debe pasar antes de que el usuario pueda cambiarla nuevamente (0).

5. **Maximum password age:** El número máximo de días que deben transcurrir antes de que se requiera un cambio de contraseña (99999).
6. **Password warning period:** El número de días antes de que caduque la contraseña, durante los cuales se advierte al usuario que se debe cambiarla (7).
7. **Password inactivity period:** El número de días después de que caduca una contraseña durante el cual el usuario debe actualizarla. Después de este período, si el usuario no cambia la contraseña, la cuenta se deshabilitará.
8. **Account expiration date:** La fecha, como número de días desde el 01/01/1970 en que se deshabilitará la cuenta de usuario. Un campo vacío significa que la cuenta de usuario nunca caducará.
9. **A reserved field:** Un campo reservado para uso futuro.

El archivo /etc/gshadow

/etc/gshadow es un archivo legible solo por root y por usuarios con privilegios de root que contiene contraseñas cifradas para grupos, cada uno en una línea separada:

```
developer:$6$7QUIhUX1WdO6$H7kOYgsboLkDseFHpk04lwAtweSUQHIpoxIgo83QND
xYtYwgmZTCU0qSCuCKErmyR263rvHiLctZVDR7Ya9Ai1::
```

Cada línea constante de **4 campos** delimitados por "dos puntos":

1. **Group name:** El nombre del grupo (developer).
2. **Encrypted password:** La contraseña cifrada para el grupo (se usa cuando un usuario que no es miembro del grupo, desea unirse al grupo usando el comando newgrp). Si la contraseña comienza con ! nadie puede acceder al grupo con newgrp).
3. **Group administrators:** Una lista delimitada por comas de los administradores del grupo (pueden cambiar la contraseña del grupo y pueden agregar o eliminar miembros del grupo con el comando gpasswd).
4. **Group members:** Una lista delimitada por comas de los miembros del grupo.

Nombre	Archivos de administración y control de usuarios
.bash_logout	Se ejecuta cuando el usuario abandona la sesión.
.bash_profile ó .profile	Se ejecuta cuando el usuario inicia la sesión. Se ejecuta solo cuando el usuario inicia sesión en el sistema.
.bashrc	Se ejecuta cuando el usuario inicia la sesión. Se ejecuta cada vez que se abre una nueva ventana de terminal o se inicia un nuevo shell Bash
/etc/group	Grupos y sus usuarios.
/etc/gshadow	Contraseñas encriptadas de los grupos.

<code>/etc/login.defs</code>	Variables que controlan los aspectos de la creación de usuarios. Se utiliza para establecer algunos atributos básicos del usuario de forma predeterminada al crear un usuario en Linux, como especificar el rango de UID y GID del usuario, el tiempo de vencimiento del usuario, la longitud máxima de la contraseña, etc.
<code>/etc/passwd</code>	Usuarios del sistema.
<code>/etc/shadow</code>	Contraseñas encriptadas y control de fechas de usuarios del sistema.

Archivos de configuración

Existen diferentes archivos de configuración (guiones shell) que se ejecutan por el shell cuando se lanza o inicia, son los archivos de arranque. En el directorio `/etc/skel` se guardan unos archivos de configuración del shell, los cuales se copian al directorio HOME asignado cuando se crea una cuenta de usuario. Estos guiones tienen como objetivo establecer la configuración de entorno del shell definiendo variables, funciones, alias, etc. Los archivos de arranque ejecutados van a depender del tipo de shell. `.bashrc`, `.bash_profile` o `.profile` (recuerda que son ficheros ocultos por comenzar por punto y para mostrarlos es necesario la opción `-a`, `ls -a`).

.bashrc

El archivo `.bashrc` está diseñado para personalizar el comportamiento de las sesiones interactivas del shell Bash. Siempre que el usuario inicia un nuevo shell interactivo (por ejemplo, al abrir una nueva ventana de terminal o conectarse a través de SSH), el sistema utiliza los comandos y las configuraciones especificadas en el archivo `.bashrc`.

Algunos casos de uso cotidiano de `.bashrc` incluyen los siguientes:

- Definición de alias de comandos.
- Configurar la apariencia del indicador de shell.
- Establecer variables de entorno.
- Establecer otras preferencias para la experiencia interactiva de un individuo.

.profile_bash ó .profile

El archivo `.bash_profile` ó `.profile` se utiliza a menudo para configuraciones relacionadas con el proceso de inicio de sesión. En la mayoría de los casos, se utiliza solo cuando un usuario inicia sesión en el sistema. Puede utilizar este archivo para establecer variables de entorno que deben persistir durante toda la sesión del usuario o ejecutar comandos que deben ejecutarse solo una vez durante el inicio de sesión. Por ejemplo, configurar la variable de entorno **PATH**.

4. Configurando cuentas de usuario

Existen herramientas gráficas para realizar todas las operaciones de configuración de cuentas de usuario, sin embargo, éstas varían de una a otra distribución Linux. Por eso en este apartado se explican **las herramientas de consola que son estándares**.

4.1. Añadir usuarios

Para añadir usuarios se emplea la utilidad **useradd** (en algunas distribuciones de sistemas Linux se usa también **adduser**). Su sintaxis básica es:

```
useradd [-c comentario] [-d home-dir] [-e fecha-expiración]
        [-f días-inactivos] [-g grupo-por-defecto] [-G grupo[,...]]
        [-m [-k directorio-esqueleto] | -M] [-p password] [-s shell]
        [-u UID [-o]] [-r] [-N] nombreusuario
```

Algunos de esos parámetros sólo son válidos cuando el sistema usa *shadow passwords*. Aunque esa es la configuración estándar de la mayoría de sistemas Linux.

En su forma más simple, sólo hay que teclear `sudo useradd -m nombreusuario` para tener una cuenta lista para funcionar. El resto de opciones sirve para modificar el comportamiento por defecto de **useradd** que se define en el archivo `/etc/login.defs`.

Justo después de crear una cuenta de usuario el administrador del sistema debería ejecutar la orden **passwd** para asignarle una contraseña como se verá en la siguiente sección. Si no se le configura una contraseña, la cuenta estará bloqueada y el usuario no podrá iniciar sesión en el sistema.

Los parámetros de la orden **useradd** tienen el siguiente significado:

- **-c comentario**. Introduce comentario en el campo Gecos del archivo `/etc/passwd` (el 5º campo de ese archivo). El nombre Gecos viene de General Electric Comprehensive Operating System, e incluye información para facilitar la identificación del usuario. Por ejemplo, en Ubuntu, por defecto ese campo se rellena con el nombre completo del usuario.

La orden **finger** sirve para pedir información sobre un usuario. Entre otras cosas muestra el contenido del campo Gecos.

```
fconde@iMac21:~$ finger fconde
```

```
Login: fconde           Name: Francisco de Asís Conde Rodríguez
Directory: /home/fconde      Shell: /bin/bash
```

- **-d home-dir**. Permite especificar el directorio que será el directorio raíz del usuario. Por defecto, si no se especifica esta opción la cuenta se creará en `/home/nombreusuario`.

- **-e fecha-expiración.** Permite introducir la fecha en la que la cuenta dejará de estar activa (a partir de esa fecha, la cuenta no se borra, pero el usuario no podrá acceder a ella). La fecha se expresa en el formato `YYYY-MM-DD`. Esta opción es muy apropiada para crear **cuentas temporales**, por ejemplo, las de empleados que se contratan sólo por un tiempo en la empresa.
- **-f días-inactivos.** Una cuenta se deshabilita si pasa un determinado número de días desde que su *password* caducó. La opción **-f** permite especificar para una cuenta concreta qué número de días deben pasar antes de que se deshabilite cuando el password ha caducado. La opción por defecto es **-1**, lo que indica que una cuenta nunca se deshabilita aunque su password haya caducado.
- **-g grupo-por-defecto.** Especifica el grupo por defecto para el usuario que se está creando. **En Ubuntu, el grupo por defecto es un nuevo grupo con el mismo nombre** que el usuario que se crea. Este comportamiento por defecto varía de una a otra distribución de Linux.
- **-G grupo[,...].** La opción **-G** permite especificar grupos adicionales a los cuales pertenece el usuario. Es una lista separada por comas.
- **-m [-k directorio-esqueleto].** La opción **-m** dice al sistema que **cree automáticamente el directorio raíz del usuario**. Normalmente la orden `useradd` lo que hace es copiar en el nuevo directorio el contenido de `/etc/skel` (con los permisos del nuevo usuario), que contiene los archivos que todo usuario debería tener por defecto en su directorio.

Si no se especifica la opción **-m**, se crea la cuenta de usuario pero no su directorio raíz, con lo que en realidad la cuenta no es utilizable. El usuario puede hacer login al sistema, pero como no tiene ningún directorio propio, no podría crear ningún archivo salvo en el directorio temporal `/tmp`.

Si se usa la opción **-m** se puede usar también la opción **-k** seguida de la ruta a un directorio. Esto permite usar otra plantilla para rellenar el contenido por defecto del nuevo directorio raíz del usuario en lugar de usar `/etc/skel`.

- **-M.** Esta opción no puede usarse con **-m** y **-k**. Lo que indica es que **no se cree un directorio raíz o de base** para el usuario. Esto es especialmente útil para determinadas **cuentas del sistema** que en realidad no necesitan un directorio propio en el disco.
- **-p password.** Esta opción sirve para asignar una contraseña preencriptada al usuario. No es muy útil salvo en scripts, porque primero es necesario encriptar la contraseña y después pasarla como opción. En su lugar es mucho mejor usar la orden `passwd` justo después de crear la cuenta.
- **-s shell.** Permite indicar la shell que el usuario usará por defecto. En la mayoría de las distribuciones Linux, la shell usada es `/bin/bash`, pero se puede especificar cualquier otra que el usuario quiera o incluso cualquier otro programa aunque no sea una shell.

Por ejemplo, **las cuentas del sistema** suelen incluir como shell el programa `/bin/false` ó `/sbin/nologin`. La shell `/bin/false` cierra cualquier tipo de entrada por consola, ssh, telnet y ftp al sistema. Por el contrario, `/sbin/nologin`

hace lo mismo, pero no evita el acceso vía `ftp`. Además, si un usuario tiene el primer tipo de shell directamente no podrá acceder al sistema y no le aparecerá ningún tipo de información. Mientras que con la segunda sí se puede conseguir mostrar un texto, por defecto **"This account is currently not available"**, que se configura en el fichero `/etc/nologin.txt`.

- `-u UID [-o]`. Permite especificar el UID que se quiere asignar a un usuario. Si se especifica un UID perteneciente a un usuario ya creado, se recibe un mensaje indicando que ese número de usuario ya está en uso. Si se usa la opción `-u` también se puede usar la opción `-o` que permite asignar el mismo UID de un usuario ya creado a un nuevo usuario. Esto en la práctica, hace que se tenga un único usuario pero con dos nombres distintos. **Recordemos que Linux identifica a los usuarios por su UID, no por su nombre.**

Un intruso, una de las primeras cosas que hará será crear una cuenta de usuario con `UID 0`, es decir, una cuenta de usuario que funcionará con acceso total al sistema como lo tiene el usuario `root`. Por lo tanto, **un buen administrador deberá revisar de vez en cuando el archivo `/etc/passwd` y eliminar cualquier cuenta sospechosa que tenga UID igual a 0.**

- `-r`. La opción `-r` indica que lo que se está creando es **una cuenta de sistema**, es decir, una cuenta con un `UID` menor que `UID_MIN` (constante definida en `/etc/login.defs`).
- `-N`. No crear grupo de usuario. Por defecto, muchas distribuciones Linux (entre ellas Ubuntu) crean un nuevo grupo para cada usuario con el mismo nombre del usuario y hacen que este grupo sea su grupo por defecto o grupo primario. Con esta opción, no se crea un grupo nuevo para cada usuario sino que se le asigna el grupo `100 (users)` que es un grupo genérico para todos los usuarios que no sean cuentas del sistema.

Diferencia entre `useradd` y `adduser`

La orden `useradd` es una utilidad estándar, que viene con la mayoría de las distribuciones de Linux. Se comporta de la misma manera en la mayoría de las configuraciones.

Los valores por defecto dependen de la configuración que el sistema tenga en `/etc/login.defs` y `/etc/default/useradd`. Los valores por defecto pueden ser si se crea un directorio raíz para el usuario, qué grupo primario se utiliza, etc.

En las distribuciones basadas en Debian, como Debian, Ubuntu, Linux Mint, Kali, etc., se añade la utilidad `adduser` es un **script de Perl que utiliza el binario `useradd`.**

La mayor ventaja del comando `adduser` es que **crea el directorio home (`/home/usuario/`)** del usuario de manera automática, **cosa que no hace `useradd`** (hay que usar la opción `-m`). Sin embargo, como no es un comando del core de GNU/Linux, es posible que no funcione bien en todas las distribuciones que uses.

4.2. Modificar cuentas de usuarios

Las cuentas de usuario pueden modificarse de varias formas. Una de ellas es accediendo al archivo `/etc/passwd` y editándolo, pero es mucho mejor usar las utilidades que se estudian en esta sección como `passwd`, `usermod` o `chage`.

Establecer la contraseña

Aunque la orden `useradd` incluye la opción `-p` para asignar una contraseña al usuario que se está creando, esto no es muy útil ya que requiere que la contraseña esté preencriptada. Por tanto, es mucho más rápido crear el nuevo usuario sin contraseña (sin la opción `-p`) y a continuación asignarle una contraseña con la orden `passwd`, que tiene la siguiente sintaxis:

```
passwd [-l] [-u [-f]] [-d] [-S] nombreusuario
```

La orden `passwd` no sólo sirve para cambiar la contraseña, sino que también permite realizar otras tareas de administración relacionadas con las cuentas de usuario.

- `-l`. La opción `-l` (lock) permite bloquear una cuenta. Esto se traduce en que el primer carácter de la contraseña cifrada en el archivo `/etc/shadow` es un signo de admiración (!).

Por defecto, el usuario root en Ubuntu está bloqueado, por lo que nadie puede hacer login en el sistema como root, lo que le daría acceso total al sistema. Esto es así por seguridad.

```
root:!:15655:0:99999:7:::
```

Un usuario no puede hacer login usando una cuenta bloqueada, pero sus archivos permanecen en el disco y es fácil volver a desbloquear la cuenta. Es muy útil cuando el administrador sospecha que hay actividades fraudulentas en una cuenta. Puede bloquear la cuenta mientras investiga, para que el usuario no pueda seguir accediendo sin necesidad de borrarla permanentemente.

- `-u`. La opción `-u` (unlock) permite **desbloquear una cuenta** eliminando el signo de admiración inicial en el campo contraseña del archivo `/etc/shadow`.

Por defecto la orden `useradd` crea cuentas sin password y bloqueadas. Por tanto, usar la orden `passwd -u` en una cuenta recién creada, da como resultado un error, ya que se tendría una cuenta sin password. Para forzar que se ejecute la orden se usa la opción `-f` que fuerza el desbloqueo de una cuenta aunque no tenga password.

- `-d`. **Elimina la contraseña** de una cuenta, dejándola sin password.
- `-s`. Muestra la **información relativa a la contraseña** de una cuenta concreta. Por ejemplo, si tiene password (P) o no (L), etc.

Los **usuarios normales** sólo pueden acceder a la **información de su propia cuenta** con `passwd`. Tampoco pueden ejecutar opciones como `-l`, `-u`, `-f`, `-d`. Es necesario ser **root** o ejecutar la orden `passwd` precedida de `sudo` para poder especificar estas opciones.

Cuando se ejecuta `passwd` sin opciones, el sistema pregunta al usuario que ejecuta la orden su contraseña actual y si es correcta, le pregunta la nueva contraseña dos veces (para evitar contraseñas incorrectas por errores de tecleo):

```
fconde@iMac21:~$ passwd
```

```
Cambiando la contraseña de fconde.  
(actual) contraseña de UNIX:  
Introduzca la nueva contraseña de UNIX:  
Vuelva a escribir la nueva contraseña de UNIX:
```

Como puede verse, ninguna de las contraseñas se muestra en la pantalla mientras se escribe, ya que de esta forma se impide que alguna otra persona que pueda estar mirando pueda ver la contraseña.

Si el usuario que ejecuta la orden `passwd` es `root`, no se le pide la contraseña actual antes de introducir la nueva contraseña. De esta forma el usuario `root` puede cambiar la contraseña de un usuario "normal" que la haya olvidado. De otra forma sería imposible, ya que ningún usuario (ni siquiera `root`) puede ver la contraseña de ningún usuario del sistema.

Nota: Linux acepta como **contraseñas** cualquier **combinación de letras, números y símbolos de puntuación**. En las contraseñas Linux también distingue entre mayúsculas y minúsculas.

Existen herramientas para generar contraseñas fuertes como `pwgen` o `apg` y verificadores de contraseñas seguras.

`pwgen` - generate pronounceable passwords

`apg` - generates several random passwords

Desbloqueando al usuario root

Como ya se ha indicado, el usuario `root` suele estar bloqueado por defecto por seguridad, ya que el usuario `root` tiene acceso total al sistema. Eso significa que nadie puede hacer login como `root`, aunque un usuario administrador sí que puede ejecutar algunas órdenes como `root` con `sudo`. Sin embargo, si es necesario, se puede desbloquear al usuario `root` siempre que se sea un usuario administrador del sistema.

En primer lugar, hay que abrir la consola en modo `root` con la orden:

```
$ sudo -s
```

El prompt cambia, y en lugar de terminar en el signo `$`, termina en signo de número `#`. Eso indica que estamos en modo `root`.

```
fconde@iMac21:~$ sudo -s
```

```
root@iMac21:~#
```

A continuación se ejecuta **passwd**. Como somos el usuario root, el cambio permite asignar una contraseña y desbloquear la cuenta.

A partir de ese momento se puede hacer login en el sistema como root, tanto en modo consola, como en modo gráfico interactivo.

Modificar un usuario ya creado

Para modificar una cuenta de usuario ya creada se usa la orden **usermod**, cuyas opciones son bastante parecidas a **useradd**. Las opciones más interesantes de **usermod** son:

```
usermod [-d directorio [-m]] [-G grupo[,...]] -a [-l nuevonombre]
        [-L] [-U] [-u UID [-o]] nombreusuario
```

- **-d directorio. Cambia el directorio raíz del usuario** (su directorio de inicio) por el que se especifica como argumento, pero no mueve los archivos que hubiera en el directorio raíz anterior al nuevo directorio.

Si se especifica la opción **-m**, entonces sí que se mueven los archivos al nuevo directorio.

- **-G grupo[,...]** Establece los grupos que se pasan como argumento como **grupos suplementarios de los que el usuario también es miembro**. Es una lista de grupos separados por comas y sin espacios.

La anterior lista de grupos a los que pertenecía el usuario se elimina y se sustituye por esta, por lo que hay que tener cuidado de no olvidar ninguno.

Para eso se usa la opción **-a**. Cuando se combina con la opción **-G**, la lista de grupos que se pasa como argumento **se añade** a la lista actual de grupos a los que pertenece el usuario en lugar de reemplazarla.

- **-l nuevonombre. Cambia el nombre de usuario** de la cuenta al nuevo que se pasa como argumento.

```
$ usermod conde -l fconde
```

cambia el nombre de usuario de la cuenta **conde** a **fconde**.

- **-L**. Permite **bloquear la cuenta**.
- **-U**. Permite **desbloquear la cuenta**.
- **-u UID [-o]**. Permite **cambiar el UID de una cuenta de usuario** al que se pasa como argumento. Si el UID que se pasa ya está asignado a una cuenta de usuario el sistema da un error. Para forzar que se reutilice el mismo UID en dos cuentas distintas se usa la opción **-o**.

Hay que usar esta opción con cuidado, ya que cambiar el UID de un usuario no afecta para nada a los archivos que ya hubiese creado ese usuario que conservan el UID anterior. Si es realmente necesario hacerlo, debería seguirse por la ejecución de la orden:

```
$ chown -R nombreUsuario /home/nombreUsuario
```

Para asegurarnos de que todos los archivos de la cuenta de usuario que están en su directorio raíz cambian su propietario al nuevo UID que se le haya asignado.

Esa orden no cambia aquellos archivos que puedan estar en otros lugares distintos del directorio raíz del usuario. Si existieran esos archivos deberían localizarse con la orden `find` y cambiarse con `chown` para evitar efectos no deseados.

Cambiar las características de una cuenta de usuario (como por ejemplo el directorio raíz) mientras el usuario está conectado al sistema puede tener consecuencias no deseadas. Por eso, un administrador, debería ejecutar la orden `who` (que indica los usuarios que están conectados en un instante dado) antes de realizar cambios en una cuenta para comprobar que el usuario al que afectan los cambios no está conectado.

La orden `chage`

Esta orden permite **modificar los ajustes de una cuenta de usuario relativos a la caducidad de la cuenta**. Es posible configurar las cuentas de usuario en Linux de forma que caduquen si:

- La contraseña no se ha cambiado en un periodo de tiempo específico.
- La fecha actual del sistema es posterior a una fecha de caducidad prefijada.

Estos ajustes son controlados por medio de la orden `chage` que tiene la siguiente sintaxis:

```
chage [-l] [-m minnumdias] [-M maxnumdias] [-d ultimodia]
      [-I diasinactividad] [-E fechacaducidad] [-W diasaviso]
      nombreusuario
```

Que tienen el siguiente significado:

- `-l`. Hace que se **muestre la información relativa a la cuenta** cuyo nombre de usuario se pasa como argumento.

```
fconde@iMac21:~$ chage -l fconde
```

```
Último cambio de contraseña : oct 27, 2012
```

```
La contraseña caduca : nunca
```

```
Contraseña inactiva : nunca
```

```
La cuenta caduca : nunca
```

```
Número de días mínimo entre cambio de contraseña : 0
```

```
Número de días máximo entre cambio de contraseñas : 99999
```

Número de días de aviso antes de que expire la contraseña : 7

- **-m minnumdias.** Establece el **mínimo número de días entre dos cambios de contraseña**. Un valor de 0 (el valor por defecto) indica que el usuario puede cambiar la contraseña tantas veces al día como quiera. Un valor de 1 indica que el usuario puede cambiar la contraseña una vez al día. Un valor de 2 indica que el usuario puede cambiar la contraseña una vez cada dos días, etc.
- **-M maxnumdias.** Establece el **máximo número de días entre dos cambios de contraseña**. Un valor de 30 indica que el usuario debe cambiar la contraseña al menos una vez cada 30 días. El valor por defecto es de 99999 que equivale aproximadamente a 270 años, lo que en la práctica equivale a que el sistema no le va a obligar a cambiarla nunca a menos que el usuario quiera cambiarla por propia iniciativa.

Si el usuario cambia la contraseña antes de que se agote el periodo de tiempo especificado, el sistema reinicializa la cuenta atrás al máximo número de días.

- **-d ultimodia.** Establece el **último día en que la contraseña fue cambiada**. Normalmente se mantiene automáticamente por las herramientas del sistema, pero puede cambiarse a mano para alterar artificialmente la cuenta de días desde el último cambio de contraseña.
- **-I diasinactividad.** **Si la contraseña ha caducado y el usuario no la cambia, al cabo de unos días la cuenta se bloquea**, de forma que es obligatorio que el usuario cambie la contraseña antes de poder utilizarla de nuevo. Este parámetro permite decidir el número de días que pasan desde que la contraseña caduca hasta que la cuenta se bloquea.
- **-E fechacaducidad.** Establece la **fecha en la que la cuenta va a dejar de estar activa**. Tiene formato YYYY/MM/DD.
- **-W diasaviso.** Establece con cuantos días de antelación se va a **avisar al usuario de que su contraseña va a caducar**. Es una buena práctica que el sistema avise al usuario de que su contraseña va a caducar para que el usuario pueda cambiar la contraseña y que su cuenta no se bloquee. El valor por defecto es una semana (7 días).

Obviamente la orden `chage` sólo puede ser ejecutada por root, excepto la opción `-1` aplicada a la propia cuenta del usuario que ejecuta la orden.

4.3. Eliminar usuarios

Cuando ya no se va a necesitar mas una cuenta de usuario, lo mejor es eliminarla del sistema. **Tener cuentas operativas pero en desuso puede plantear problemas de seguridad**, por ello el administrador del sistema, de vez en cuando debe eliminar aquellas cuentas que ya no sean necesarias.

La orden para eliminar cuentas de usuario es `userdel` cuya sintaxis es la siguiente:

```
userdel [-r [-f]] nombreusuario
```

Las opciones tienen el siguiente significado:

- **-r. Elimina también los archivos del usuario** que están en su directorio raíz `/home/nombreusuario` y el propio directorio raíz, así como el correo del usuario. **Si no se especifica** la opción `-r`, la cuenta de usuario se elimina pero no sus archivos, lo que puede dar lugar a confusión.

Si dentro del directorio `/home/nombreusuario` hay archivos que no son propiedad de la cuenta `nombreusuario`, entonces esos archivos permanecen en el disco. La opción `-f` que sólo se puede usar junto con `-r`, fuerza a eliminar todos los archivos del directorio `/home/nombreusuario` tanto si son propiedad de `nombreusuario` como si no.

En ocasiones puede ser interesante hacer una copia de respaldo de los archivos de un usuario antes de eliminarlos del directorio `/home`. De esta manera se conserva el trabajo hecho por un empleado que ya no pertenezca a la empresa, etc. Para ello se usa la orden `find` ejecutada como usuario `root`.

Por ejemplo, si se quiere hacer una copia de todos los archivos del usuario `user1` en la carpeta del usuario `root` (donde sólo él puede leerlos) antes de eliminar la cuenta de `user1` la orden es¹:

```
$ sudo find / -user user1 -exec cp {} /root/user1 \;
```

Si se olvidó usar la opción `-r` al eliminar una cuenta de usuario, ya no es posible usar el nombre de usuario para localizar sus archivos con la orden `find` (opción `-user nombreusuario`), en su lugar, hay que usar la opción `-uid id`, donde `id` es el identificador del usuario cuyos archivos se quieren eliminar.

Si no nos acordamos del `id` del usuario que hemos borrado, siempre se puede ir a la carpeta `/home/nombreusuario` y allí con la opción `ls -l` ver a quién está asignado alguno de los archivos que contenga.

5. La órdenes `su` y `sudo`

Algunos usuarios noveles, creen que "`su`" es un diminutivo de "`sudo`" y que hacen lo mismo. Estas órdenes son totalmente distintas y cada una sirve para una cosa muy distinta.

La orden "`su`"

¹ `-exec command` Realiza una búsqueda y le aplica la orden especificada a cada fichero, `{}` lo reemplaza por cada fichero y la barra invertida punto y coma `\;` indica el final de la instrucción, fin de `-exec`. El carácter `\` es necesario para escapar el punto y coma (`;`), que es parte de la sintaxis del comando `find`, de esa forma se evita que lo interprete el shell.

Son las siglas de "**Switch User**" y sirve, como su propio nombre indica, para **cambiar de usuario sin necesidad de hacer un cierre o cambio de sesión** (todo esto desde la terminal, esa cosa tan rara, pero potente y funcional).

Vamos a ver unos ejemplos y para ello, vamos a utilizar dos órdenes más: **whoami** (quien soy) y **pwd** (directorio en el que nos encontramos en la terminal):

```
lina@equipo:~$ whoami
```

```
lina
```

```
lina@equipo:~$ pwd
```

```
/home/lina
```

Ahora vamos a cambiar al usuario "fconde" con el comando: "su nombreusuario":

```
lina@equipo:~$ su fconde
```

```
Contraseña: [la de fconde]
```

Y comprobamos quién es y dónde está:

```
fconde@equipo:/home/lina$ whoami
```

```
fconde
```

```
fconde@equipo:/home/lina$ pwd
```

```
/home/lina
```

Vemos que es "fconde" pero sigue en mi carpeta personal "/home/lina" y no en la suya, **no cambiando las variables de entorno**. Eso sí, en el Prompt (texto por defecto antes del "\$"), esta variable de entorno es actualizada y nos muestra el nuevo usuario y el home del usuario, donde nos encontramos: "**fconde@equipo:/home/lina\$**". Esto es muy importante tenerlo en cuenta a la hora de hacerse root.

Si queremos cambiar al usuario "manuel" y además cambiar a su carpeta personal y demás variables de entorno, añadimos un espacio en blanco y un guión medio "-" a "su" (antes de fconde):

```
lina@equipo:~$ su - fconde
```

```
Contraseña: [la de fconde]
```

```
fconde@equipo:~$ whoami
```

```
fconde
```

```
fconde@equipo:~$ pwd
```

/home/fconde

La orden "sudo"

"Sudo" viene de "*switch user do*" y permite a los usuarios ejecutar acciones con los privilegios de seguridad del root, de manera segura. Se instala por defecto en `/usr/bin`.

Por defecto Ubuntu trae desactivada la cuenta del "root", por seguridad y para administrar el sistema existe un grupo de usuarios denominado "**sudoers users**" (administradores o admin), los cuales pueden obtener permisos de root, mediante la utilización de "**sudo**".

El usuario con el que instalamos Ubuntu, se encuentra incluido en este grupo de administradores.

El archivo que determina el acceso a las tareas administrativas del grupo "**sudoers users**" es `/etc/sudoers`. En Ubuntu 24.04 el archivo por defecto viene configurado así:

```
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults    env_reset
Defaults    mail_badpass
Defaults
secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"
# This fixes CVE-2005-4890 and possibly breaks some versions of kdesu
# (#1011624, https://bugs.kde.org/show_bug.cgi?id=452532)
Defaults    use_pty

# This preserves proxy settings from user environments of root
# equivalent users (group sudo)
#Defaults:%sudo env_keep += "http_proxy https_proxy ftp_proxy all_proxy
no_proxy"

# This allows running arbitrary commands, but so does ALL, and it means
# different sudoers have their choice of editor respected.
#Defaults:%sudo env_keep += "EDITOR"

# Completely harmless preservation of a user preference.
#Defaults:%sudo env_keep += "GREP_COLOR"

# While you shouldn't normally run git as root, you need to with etckeeper
#Defaults:%sudo env_keep += "GIT_AUTHOR_* GIT_COMMITTER_*"

# Per-user preferences; root won't have sensible values for them.
#Defaults:%sudo env_keep += "EMAIL DEBEMAIL DEBFULLNAME"

# "sudo scp" or "sudo rsync" should be able to use your SSH agent.
#Defaults:%sudo env_keep += "SSH_AGENT_PID SSH_AUTH_SOCK"

# Ditto for GPG agent
#Defaults:%sudo env_keep += "GPG_AGENT_INFO"
```

```
# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@include /etc/sudoers.d
```

Veamos algunos de los formatos y reglas a seguir al editar **sudoers**:

- Todas las líneas que comienzan con **#** son comentarios
- **root ALL=(ALL: ALL) ALL** – esta línea significa que el usuario **root** tiene privilegios ilimitados y puede ejecutar cualquier comando en el sistema
- **%admin ALL=(ALL) ALL** – el signo **%** especifica un grupo. Cualquier persona en el grupo de administradores tiene los mismos privilegios que el usuario **root**
- **%sudo ALL = (ALL: ALL) ALL** – todos los usuarios del grupo **sudo** tienen los privilegios para ejecutar cualquier comando

Otra línea de interés es **#includedir /etc/sudoers.d**, esta significa que podemos agregar configuraciones al archivo **sudoers.d** y vincularlo con este.

Este **archivo configura el sistema de forma que cualquier usuario del grupo "admin"** (administrador) pueda realizar tareas administrativas anteponiendo el **"sudo"** a cualquier orden o comando.

En la terminal se utiliza la orden **"sudo"**, anteponiéndolo a la orden a ejecutar:

sudo orden

Para ver el manual de **"sudo"** ejecutar en una terminal:

\$ man sudo

Para salir del manual pulsar la tecla **q**.

En Ubuntu, la mejor opción es usar la orden **sudo** en cada uno de las órdenes que queremos ejecutar:

sudo orden

Pero si hay que ejecutar muchos podemos usar:

```
$ sudo -i
```

Mejor que entrar directa o indirectamente con la cuenta "root".

6. Configurando grupos

Las órdenes para configurar grupos tienen una estructura muy parecida a las órdenes para configurar usuarios. También existen herramientas gráficas para hacer el trabajo, pero, al igual que pasaba con las herramientas para usuarios, no están estandarizadas entre las distintas distribuciones de Linux, por lo que en esta sección se van a estudiar las herramientas de línea de órdenes.

6.1. Añadir grupos

Linux proporciona la orden **groupadd** para añadir un nuevo grupo al sistema. Es necesario ser `root` para ejecutar esta orden o usar **sudo**. Es parecida a la orden **useradd**, pero tiene menos opciones. Su sintaxis es:

```
groupadd [-g GID [-o]] [-r] nombregrupo
```

El significado de cada opción es el siguiente:

- **-g GID [-o]**. Permite elegir un identificador concreto para el grupo: `GID`. Si se omite este parámetro, el sistema automáticamente asigna el siguiente identificador que esté disponible mayor que `GID_MIN` (esta constante se define en el archivo `/etc/login.defs`)

Si se intenta crear un grupo con el mismo identificador de otro grupo ya creado, el sistema dará un error. Para forzar que se use el mismo identificador para el nuevo grupo se usa la opción **-o** (sólo se puede usar junto con la opción **-g**). En la práctica, se tiene un único grupo, pero con varios nombres.

- **-r**. Indica a **groupadd** que cree un grupo de sistema en lugar de un grupo "normal" de usuario.

Una **cuenta de sistema**, tiene identificador menor que `GID_MIN` (definido en `/etc/login.defs`)

En la mayoría de los casos, no será necesario especificar ninguna de las opciones.

Por ejemplo, para crear un grupo de usuario llamado **proyecto1**, la orden es:

```
$ sudo groupadd proyecto1
```

6.2. Modificar grupos

La información de un grupo puede ser modificada una vez que éste se ha creado. Para ello se usan dos órdenes: **groupmod** y **usermod**.

La orden **groupmod** tiene la siguiente sintaxis:

```
groupmod [-g GID [-o]] [-n nuevonombregrupo] nombregrupo
```

El significado de las opciones es el siguiente:

- **-g GID [-o]**. Cambia el identificador del grupo por el nuevo que se pasa como argumento. Si ya existe un grupo con el nuevo identificador que se quiere asignar al grupo, el sistema devuelve un mensaje de error. La opción **-o** (sólo se puede usar junto con **-g**) fuerza que se reutilice un identificador ya en uso en otro grupo. Esto es peligroso porque crea una colisión de GID y puede llevar a problemas de permisos y seguridad. Por ejemplo para cambiar el ID de un grupo a uno no usado:

```
$ sudo groupmod -g 800 administradores
```

- **-n nuevonombregrupo**. Permite cambiar el nombre del grupo por el nuevo que se pasa como argumento. Por ejemplo, el grupo se llama desarrollo y los renombra como programación

```
$ sudo groupmod -n programacion desarrollo
```

Uno de los cambios que más frecuentemente se realizan a los grupos, que consiste en añadir y eliminar usuarios que pertenecen al mismo, se realiza con la orden **usermod** que ya se ha estudiado. En concreto, la opción **-G** permite añadir a un usuario a un grupo. Se pueden especificar varios grupos separados por comas sin espacios.

La siguiente orden (ejecutada como root):

```
$ sudo usermod -G grp1,grp2,grp3 usuario1
```

Añade al usuario: `usuario1`, a los grupos ya creados `grp1`, `grp2` y `grp3`, y lo elimina de los otros grupos a los que pudiera pertenecer.

Es muy importante asegurarse de que en la opción **-G** se escriben todos los grupos a los que se quiera que pertenezca el usuario, ya que el usuario dejará de pertenecer a aquellos que no se escriban. Para averiguar los grupos a los que pertenece un usuario se puede usar la orden:

```
$ groups nombreusuario
```

También se pueden ver en el archivo `/etc/group`. Por ejemplo, la siguiente orden muestra todos los grupos a los que pertenece el usuario `fconde` en un sistema Linux:

```
fconde@iMac21:/home$ cat /etc/group | grep fconde
```



```
adm:x:4:fconde
cdrom:x:24:fconde
sudo:x:27:fconde
dip:x:30:fconde
plugdev:x:46:fconde
lpadmin:x:109:fconde
fconde:x:1000:
sambashare:x:124:fconde
pepe:x:1002:fconde
```

que en este ejemplo concreto son: `adm`, `cdrom`, `sudo`, `dip`, `plugdev`, `lpadmin`, `fconde`, `sambashare` y `pepe`.

Muchos administradores prefieren usar la orden **`gpasswd`** en lugar de **`usermod`** para **añadir usuarios a grupos**, ya que permite añadir nuevos usuarios a grupos sin necesidad de escribir todos los grupos a los que ya pertenezcan esos usuarios. Lo que es menos propenso a errores.

Una de las órdenes más potentes para modificar grupos es la orden **`gpasswd`**. Su sintaxis es:

```
gpasswd [-a user] [-d user] [-R] [-r] [-A usuario1[,...]]
          [-M usuario1[,...]] grupo
```

El significado de las opciones es el siguiente:

- **-a nombreusuario**. Añade el usuario que se indica como nuevo miembro del grupo, dejando el resto de miembros de ese grupo inalterados.

Esta opción suele ser preferible a la opción **-G** de la orden `usermod` para añadir usuarios a grupos.

- **-d nombreusuario**. Elimina el usuario que se indica como miembro del grupo. A partir de ese momento el usuario ya no pertenece al grupo.
- **-R**. Impide el uso de la orden **`newgrp`** en este grupo.
- **-r**. Elimina la contraseña para el grupo.
- **-A usuario1[,...]**. Sustituye la lista de usuarios administradores de este grupo por la que se pasa como argumento. Es una lista de nombres de usuarios separados por comas y sin espacios.

Un usuario administrador de un grupo puede añadir y eliminar miembros a ese grupo, así como cambiar su contraseña.

El uso de esta opción sobrescribe completamente el contenido de la lista de administradores del grupo, por lo que hay que asegurarse de que se escribe el nombre de usuario de todos los administradores.

- **-M usuario1[,...]**. Funciona igual que la opción **-A**, pero además añade a los usuarios como miembros del grupo.

Si se usa la orden `gpasswd` sin ninguna opción, entonces la orden permite cambiar la contraseña para un grupo.

La contraseña de un grupo afecta a la orden `newgrp`, que como sabemos, permite que un usuario miembro de un grupo, pueda hacerlo temporalmente su grupo primario, de forma que a partir de ese momento, todos los archivos que cree o los programas que ejecute estén asignados a ese grupo. Si un grupo tiene contraseña, los usuarios pertenezcan al grupo pueden seguir usando la orden `newgrp` sin utilizar la contraseña, y a los usuarios que no pertenezcan al grupo el sistema le va a pedir la contraseña.

Sin embargo, en Ubuntu moderno, las contraseñas de grupo están prácticamente obsoletas y rara vez se usan. La función principal de `newgrp` hoy en día es permitir a un usuario miembro cambiar su GID efectivo para la sesión actual.

6.3. Eliminar grupos

Para eliminar un grupo se usa la orden `groupdel`, cuya sintaxis es muy sencilla:

```
groupdel nombregrupo
```

Si el grupo que se quiere eliminar es el grupo primario de algún usuario, el sistema informa del error y no permite que se borre al grupo. Antes habría que **cambiar el grupo primario del usuario a otro grupo para poder eliminar el grupo**.

De la misma forma que ocurría cuando se eliminaba un usuario, **si se elimina un grupo, se debe localizar todos los archivos asignados a ese grupo mediante su GID**. Para ello se usa la orden `find` con la opción `-gid`.

Por ejemplo, la orden:

```
find / -gid 1000
```

busca en el sistema completo todos los archivos cuyo identificador de grupo GID es igual a 1000.

Procedimiento Post-Eliminación

Una vez localizados los archivos con el GID antiguo, un administrador puede:

1. **Reasignar la Propiedad:** Cambiar la propiedad del grupo de esos archivos a un grupo existente, usando `chgrp`.

Cambia el grupo propietario a 'administradores' para todos los archivos encontrados.²

² `-exec command` Realiza una búsqueda y le aplica la orden especificada a cada fichero, `{ }` lo reemplaza por cada fichero y la barra invertida punto y coma `\;` indica el final de la instrucción, fin de `-exec`. El carácter `\` es necesario para escapar el punto y coma (`;`), que es parte de la sintaxis del comando `find`, de esa forma se evita que lo interprete el shell.

```
$ sudo find / -gid 1000 -exec chgrp administradores {} \;
```

2. **Eliminar los Archivos:** Si los archivos ya no son necesarios, se pueden eliminar.

Ejemplos Gestión de Cuentas

1. **Añadir la cuenta** con nombre `Ada Lovelace`, login `alovelace`, shell `bash` al sistema.

```
$ sudo useradd -c "Ada Lovelace" -s /bin/bash -m alovelace
```

```
$ cat /etc/passwd | grep alovelace
```

```
alovelace:x:1002:1002:Ada Lovelace:/home/alovelace:/bin/bash
```

Ahora mismo la cuenta está bloqueada porque no tiene contraseña

```
$ sudo cat /etc/shadow | grep alovelace
```

```
alovelace:!:19281:0:99999:7:::
```

2. **Poner una contraseña** a la cuenta de Ada Lovelace

```
$ sudo passwd alovelace
```

```
Enter new UNIX password: maths_2022
```

```
Retype new UNIX password: maths_2022
```

```
passwd: password updated successfully
```

3. **Comprobar la cuenta** de Ada Lovelace

```
$ su alovelace (Password: maths_2022 )
```

```
$ id
```

```
uid=1002(alovelace)
```

```
gid=1002(alovelace)
```

```
groups=1002(alovelace)
```

4. **Añadir el grupo** `babbage`

```
$ sudo groupadd babbage
```

```
$ cat /etc/group | grep babbage
```

```
babbage:x:1007:
```

5. **Cambiar la shell** de Ada Lovelace por `tcsh`

```
$ sudo usermod -s /bin/tcsh alovelace
```

```
$ cat /etc/passwd | grep alovelace
```

```
alovelace:x:1002:1002:Ada Lovelace:/home/alovelace:/bin/tcsh
```

6. Añadir a Ada Lovelace al grupo `babbage` además de al grupo `alovelace`

```
$ cat /etc/group | grep babbage
```

```
babbage:x:1007:
```

```
$ sudo usermod -a -G alovelace,babbage alovelace
```

```
$ cat /etc/group | grep babbage
```

```
babbage:x:1007:alovelace
```

7. Revisar la información de la cuenta

```
$ sudo chage -l alovelace
```

```
Last password change : Nov 19, 2022
Password expires : never
Password inactive : never
Account expires : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

8. Establecer la fecha de caducidad de la cuenta de Ada Lovelace en el 15 de diciembre de 2025, el número mínimo de días antes del cambio de contraseña en diez y el número máximo de días antes del cambio de contraseña en veinte.

```
$ sudo chage -E 2025-12-15 -m 10 -M 20 alovelace
```

Ejercicios

1. Revisa el archivo `/etc/passwd` y escribe la lista de los usuarios definidos en el mismo que tengan como shell `/bin/false`. ¿Qué tipo de usuarios son? ¿Por qué tienen esa shell asignada?

2. Visualiza el contenido de los archivos `/etc/default/useradd` y `/etc/login.defs`. Comprueba cuáles son las opciones por defecto que tendría un usuario que se creara en tu sistema. A continuación, crea una cuenta de usuario y visualiza el contenido de los archivos `/etc/passwd`, `/etc/group`, y el directorio `/home` para comprobar que los nuevos datos se han rellenado conforme a la especificación tomada de `/etc/default/useradd` y `/etc/login.defs`.

3. Supongamos que se tiene un disco duro auxiliar montado en el directorio `/home2`. Supongamos que existen los grupos de usuarios `depto1` y `depto2`.

Se quiere crear una nueva cuenta de usuario completamente operativa para un usuario llamado Juan López (debe tener una contraseña), cuya directorio raíz deberá estar ubicado en `/home2`, que pertenezca a los grupos `depto1` y `depto2` siendo su grupo primario `depto1` y con shell por defecto `tcsh`.

¿Cual es la/s orden/es que debes escribir para conseguirlo?

4. Siendo el usuario `root`, se quiere eliminar la cuenta del usuario `user1` cuyo directorio raíz es `/home/user1` y que además tiene algunos archivos en el directorio `/tmp`.

Escriba las órdenes necesarias para eliminar completamente la cuenta del usuario `user1` incluyendo todos sus archivos, pero haciendo previamente una copia de seguridad de los mismos en el directorio `/root`.

5. Averigua a qué grupos pertenece el usuario con el que te hayas conectado para hacer las prácticas. Hazlo al menos de dos formas distintas. Explica cómo lo has hecho.

6. ¿Qué dos formas existen en Linux para añadir, con una orden, un usuario a un grupo ya creado? ¿Cuál de ellas es mejor? ¿Por qué?

7. Un usuario del sistema con nombre de usuario `empleado1` ha olvidado su contraseña. ¿Qué harías para restablecerla?

8. ¿Para qué se usa la contraseña de un grupo?

9. Si se pudiese eliminar un grupo que sea el grupo primario de algún usuario, el sistema quedaría en una situación de funcionamiento inaceptable. ¿Cual es esa situación de funcionamiento? ¿Por qué ocurre esto?

10. Averigua, ejecutando una orden, qué UID y GID mínimo y máximo se han definido en Linux Ubuntu (debes usar un pipeline).

Preguntas frecuentes

Usuario root ¿Cómo pasar a ser usuario root?

Aunque utilizando `sudo` podemos ejecutar cualquier comando con permisos de administrador, a veces necesitamos ejecutar múltiples comandos y es un poco tedioso tener que estar poniendo `sudo` delante de cada uno de ellos y luego introducir de nuevo la contraseña de nuestro usuario. Para hacer el proceso más cómodo, podemos hacer login directamente como root con el comando `sudo -i`:

```
$ sudo -i
```

```
[sudo] password for mario:
```

```
root@geekytheory:~# whoami
```

```
root
```

```
root@geekytheory:~#
```

Es importante fijarse en que cuando estamos utilizando la consola como usuario root, el último carácter antes de introducir nuestro comando es una almohadilla `#`.

Por otra parte, para volver a utilizar nuestro usuario basta con introducir el comando `exit`.

Cuenta root ¿Es necesario activar la cuenta root en Ubuntu?

Los desarrolladores de Ubuntu Linux en un principio querían que el proceso de instalación fuese lo más fácil posible, el hecho de desactivar de manera predeterminada la cuenta de usuario `root` permitía obviar algunos pasos en el proceso de instalación. Esto para muchos es un inconveniente, pues Ubuntu Linux “difiere” en cuanto al modelo de seguridad que se maneja comúnmente en sistemas GNU/Linux, el modelo que plantea Ubuntu Linux es simplemente no recomendar hacer uso extensivo del usuario `root` (por eso ha desactivado la cuenta). Debido a que esta distribución está orientada hacia usuarios que quizás no han tenido un contacto extenso con sistemas GNU/Linux, el modelo propuesto me parece bastante lógico.

Algunas personas dicen que el usar `sudo` es tedioso, puesto que debes introducir la contraseña a cada instante. Esto no es del todo cierto, la primera vez que introduces la contraseña, ésta se almacenará por quince (15) minutos, después de transcurrido ese tiempo y si necesitas hacer alguna actividad administrativa se te volverá a solicitar.

Quizás la mayor desventaja de este modelo es que el mantener una contraseña “diferente” para el superusuario existe menor protección en el caso en que las contraseñas de los usuarios con derechos administrativos se vean comprometidas. Pero esto puede ser evitado al ponerle mayor cuidado a las cuentas de usuarios con derechos administrativos, la debilidad está ahí y no en el modelo en cuestión.

Lo cierto es que el uso de `sudo` puede considerarse para ejecutar pocos comandos administrativos, mientras que `su` generalmente es utilizado para ejecutar múltiples tareas administrativas, el problema es que `su` puede dejar “abierta” indefinidamente una shell con derechos de superusuario, esto último es un gran inconveniente para la seguridad del sistema. En cambio `sudo` limita estas cosas, como se mencionó anteriormente, al menos tienes quince (15) minutos de derechos de superusuario.

Cuenta en Ubuntu: Activar la cuenta root en Ubuntu

Ubuntu, por defecto, bloquea la cuenta root y promueve el uso de `sudo` para ejecutar comandos administrativos con la contraseña del usuario. Si un administrador quiere cambiar a un modelo tradicional (donde solo la contraseña de root funciona para las tareas críticas), debe seguir estos pasos:

Para cambiar este comportamiento el usuario puede establecer una contraseña de `root` ejecutando `sudo passwd root`, primero deberá introducir su contraseña de usuario para poder establecer una contraseña de root.

1. Activar la Cuenta root

El primer y principal paso es darle una contraseña a la cuenta, lo que la desbloquea. Para activar la cuenta root recién instalada Ubuntu se hace de la siguiente manera.

```
$ sudo -u root passwd
```

y pones la contraseña del usuario que has creado anteriormente, después de eso te pedirá el nuevo password para el root.

Con esto ya estamos en condiciones de hacer

```
$ su -
```

para entrar a nuestra cuenta de root.

2. Restaurar la Separación Tradicional de Roles

Los siguientes pasos se centran en deshabilitar la conveniencia de `sudo` para forzar el uso de la recién activada contraseña de root.

Deshabilitar sudo para Usuarios Normales

Para evitar que el usuario principal pueda seguir usando `sudo` con su propia contraseña. Si el usuario principal puede usar `sudo`, la activación de la cuenta root no logra la "separación" deseada.

Por tanto, se debe modificar el archivo `/etc/sudoers` o un archivo de configuración en `/etc/sudoers.d/` para eliminar o comentar la línea que otorga privilegios al grupo sudo (o al usuario). Por ejemplo, deshabilitar la línea `%sudo ALL=(ALL:ALL) ALL`.

Gestión de Aplicaciones Gráficas

La mayoría de las aplicaciones modernas de Ubuntu y su entorno gráfico gestionan los permisos a través de *PolicyKit*. Al intentar realizar una acción administrativa, el sistema te presentará un diálogo que te permite introducir la contraseña del usuario principal (el que tiene privilegios sudo) o, si has activado la cuenta root, te pedirá su contraseña si la política de seguridad así lo requiere.

Por tanto, la activación de la cuenta root es válida con `sudo passwd root`, el resto de los pasos buscan deshacer las características de conveniencia de Ubuntu para re-imponer la dependencia de la contraseña de root en todas las tareas administrativas.

Usuario root ¿Cómo deshabilitar el usuario root?

Para bloquear de nuevo el usuario root en Linux simplemente hay que introducir el siguiente comando:

```
$ sudo passwd -dl root
```